Lesson 8 Teacher's Key: Under Pressure

GameMaker - Instructor Reference Only

VIDEO GAME CREATION

What This Lesson Teaches

Extending an already-correct code path (obj_enemy's Step event, fixed in Lesson 7 to only handle movement/escape) with a second consequence routed through obj_gamemanager - reinforcing exactly why last week's fix mattered, and why Rule 13's dedicated manager object earns its keep the moment a second piece of shared state shows up.

Level 1 - Full Worked Solution

- __dnd_lives and has_lost are added to obj_gamemanager's Create event alongside enemies_remaining and has_won - same object, same pattern, different variables, all still living on the one dedicated manager instance introduced in Lesson 6.
- The lives decrement and zero-check are added directly inside the EXISTING escape branch of obj_enemy's Step event (the if (y > room_height) block), right before instance_destroy() - not a new event, just new lines in an existing, already-correct one, now reaching out to obj_gamemanager instead of obj_spawner.
- obj_gamemanager's existing Draw GUI event gets a second if-block for has_lost, alongside the existing has_won block - the same object that already draws "You Win!" now also draws "Game Over".

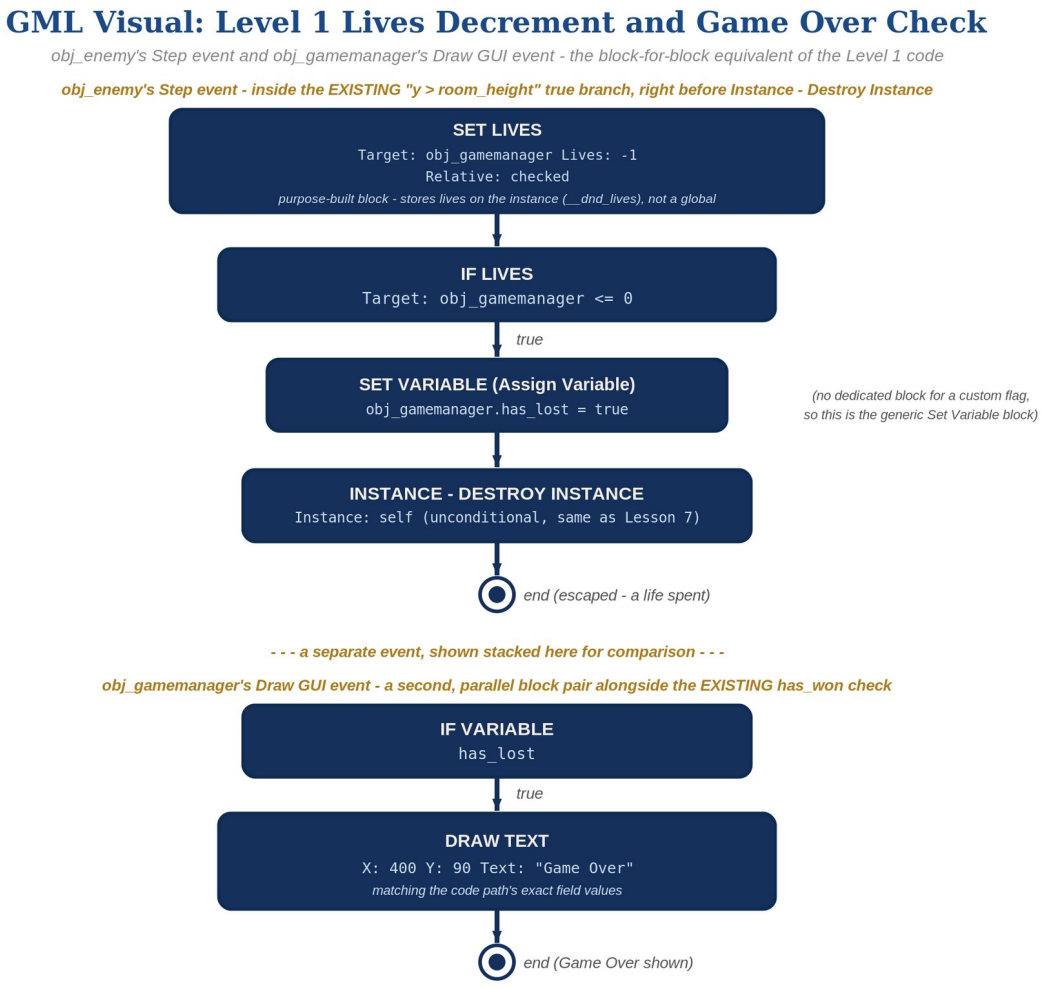
Highlighted lines are new this week; everything else is code you already have.

```
// obj_gamemanager's Create event:
enemies_remaining = 5;
has_won = false;
__dnd_lives = 3;
has_lost = false;

// obj_enemy's Step event:
y += enemy_speed;
if (y > room_height) {
    obj_gamemanager.__dnd_lives -= 1;
    if (obj_gamemanager.__dnd_lives <= 0) {
        obj_gamemanager.has_lost = true;
    }
    instance_destroy();
}

// obj_gamemanager's Draw GUI event:
if (has_won) {
    draw_text(400, 50, "You Win!");
```

```
}
if (has_lost) {
    draw_text(400, 90, "Game Over");
}
```



obj_enemy's Step event and obj_gamemanager's Draw GUI event, in GML Visual - the block-for-block equivalent of the Level 1 code.

Why This Structure

- This is the most direct possible payoff of Lesson 7's bugfix AND this week's Rule 13 retrofit: because obj_enemy's Step event was already the single, unambiguous home for "this enemy left the screen unshot," and obj_gamemanager was already the single home for shared game state, adding a new consequence required touching nothing else.
- Contrast this with what would have happened before the Rule 13 retrofit, when lives briefly lived on obj_spawner: a student following last week's habits would have had two different objects to check depending on which piece of state they wanted, instead of one.

Level 2 / Level 3 Talking Points

- Level 3 sound, worked (GameMaker): import sfx_lifelost.wav and sfx_gameover.wav keeping their default names, sfx_lifelost and sfx_gameover. Where obj_enemy's Step event does obj_gamemanager.__dnd_lives -= 1, follow with: if (obj_gamemanager.__dnd_lives <= 0) { obj_gamemanager.has_lost = true; audio_play_sound(sfx_gameover, 1, false); } else { audio_play_sound(sfx_lifelost, 1, false); }. Visual path: a Play Sound block inside the If Lives block (game over) and another after it (life lost) - use If Lives' Not option or a second If Lives (Expression greater than, Value 0) so only one sound plays.
- This is a great lesson to explicitly connect back to last week: ask students to recall why the GameMaker fix mattered, and now show them it's the reason this week was easy.
- Level 3 structure: obj_bullet's Collision event and obj_enemy's Step event remain the entire win/lose logic surface of the game so far, and obj_gamemanager remains the only object holding any of that state - worth pointing out how compact and consistent that still is, even as features accumulate.

Level 2 - complete code:

```
// Level 2 is verification only - the Level 1 code is already correct, so
no lines change.
// Checks: shooting an enemy never touches obj_gamemanager.__dnd_lives
(only obj_bullet's Collision event
// changes enemies_remaining); an escape never touches enemies_remaining;
"Game Over" draws once, when
// __dnd_lives first reaches 0.
```

Level 3 - complete code (all three goals combined):

Highlighted lines are the Level 3 additions on top of the Level 1 code.

```
// obj_gamemanager's Create event:
enemies_remaining = 5;
has_won = false;
__dnd_lives = 3;
has_lost = false;

// obj_enemy's Step event (Structure: still the only place lives change;
Sound: both clips):
y += enemy_speed;
if (y > room_height) {
    obj_gamemanager.__dnd_lives -= 1;
    if (obj_gamemanager.__dnd_lives <= 0) {
        obj_gamemanager.has_lost = true;
        audio_play_sound(sfx_gameover, 1, false);
    } else {
        audio_play_sound(sfx_lifelost, 1, false);
    }
    instance_destroy();
}

// obj_gamemanager's Draw GUI event (Visual: live lives readout, drawn
every frame):
draw_text(20, 20, "Lives: " + string(__dnd_lives));
if (has_won) {
    draw_text(400, 50, "You Win!");
}
```

```
if (has_lost) {
    draw_text(400, 90, "Game Over");
}

// Visual path alternative for the readout: replace the first draw_text()
with a Draw Instance Lives block
// (spr_player, Caption "Lives: ", X 20, Y 20) or a Draw Value block
(Caption "Lives: ", Value __dnd_lives).
```

Common Student Mistakes

- Adding the lives decrement to obj_bullet's Collision event instead of obj_enemy's Step event (makes shooting cost a life, which is backwards).
- Adding __dnd_lives/has_lost to obj_spawner instead of obj_gamemanager, out of habit from before the Lesson 7 GameManager retrofit - watch for this specifically, since it's the most likely regression this week.
- Forgetting the has_lost flag and just checking __dnd_lives <= 0 directly in Draw GUI without ever setting a flag (works, but inconsistent with the has_won pattern already established - point this out as a style/consistency issue, not a bug).
- "Game Over" and "You Win!" text overlapping on screen because both draw_text() calls use the same Y coordinate.

Alternate Path Notes: GML Visual (no typing)

obj_enemy's Step event works identically whether it holds blocks or code - the event itself already fires exactly once per instance either way, and reaches out to obj_gamemanager the same way in both paths.

The block path uses GameMaker's purpose-built Lives blocks - "Set Lives" (Relative checked, Value -1) and "If Lives" (Expression less than or equal to, Value 0) - each with its Target drop-down set to obj_gamemanager. Behind the scenes these keep an instance variable named __dnd_lives (the "dnd" is drag-and-drop; it is NOT a global). The code path deliberately uses that exact variable, so a student can mix code and blocks in one project - which is why the Target matters: left at the default, the blocks would act on the enemy that is about to be destroyed. has_lost, a custom variable with no dedicated block, still uses the generic "Set Variable" block, exactly as the code path's plain assignment does.

Create-event setup mirrors the code path: a "Set Lives" block with Value 3 and Relative unchecked on obj_gamemanager's Create event.

Level 3 visual: "Draw Instance Lives" (Drawing category, Draw GUI event, spr_player as the sprite) is the purpose-built way to show lives as icons; "Draw Value" with Value __dnd_lives is the text alternative. Draw Instance Lives only draws frame 0 of the sprite and ignores image_xscale/image_blend - fine for spr_player.

obj_gamemanager's Draw GUI event gets the same second "If Variable" + "Draw Text" block pair whether Lesson 7's win check was built in Code or Visual - same object, same event, same targeting.

- Keeping the same Structure principle (obj_gamemanager owns all shared state, obj_enemy only ever reaches out to modify it) whether a student is in Code or Visual reinforces that Rule 13 is a design idea, not a syntax rule tied to one coding style.

- A student switching between Code and Visual mid-course (or comparing notes with a classmate using the other track) should recognize obj_gamemanager playing the exact same role either way.